

SOURCE-CODE SECURITY ASSESSMENT

ngoab-main

Static Application Security Review & Research Report

Severity	Count
Critical	3
High	6
Medium	5
Low	5
Info	0
Total confirmed	19

Stack	PHP ^8.0, Laravel 9.52.21, tymon/jwt-auth 2.2.1 (HS256) mobile/API, MySQL + Redis, exceptio/laravel-sonali-payment 1.0 (Sonali Bank SPG), mPDF 8.2.7 + barryvdh/laravel-dompdf 3.1.4 + niklasravnsborg/laravel-pdf 4.1.0, maatwebsite/excel 3.1 + phpooffice/phpspreadsheet 1.30.2, opcodesio/log-viewer 3.21.1, Laravel Mix, Docker Compose
Assessment date	2026-08-24
Report generated	2026-08-24 13:29
Reviewer	Aubria
Team	SCR source-code vulnerability review team
Commit	not a git repository (working tree only)

Confidential — for the recipient team only.

Contents

Objective	3
Executive Summary	3
Methodology	3
Scope & Coverage	4
Findings Index	4
Detailed Findings	6

Objective

Identify security vulnerabilities in the target source code through static analysis, assess their real-world exploitability, and provide prioritized, actionable remediation guidance.

Goal. Deliver a defensible, reproducible assessment suitable for both engineering remediation and security research, with every finding traced to a concrete code location and mapped to CWE / OWASP taxonomies.

Executive Summary

This assessment of ngoab-main identified 19 confirmed finding(s): 3 Critical, 6 High, 5 Medium, 5 Low. The overall residual risk posture is assessed as critical, driven primarily by the highest-severity issues. A further 12 candidate(s) were investigated and dismissed with code evidence, reducing false-positive noise.

Priority issues: SCR-001 (Critical) Unauthenticated PDF/PHP-polyglot upload -> remote code execution; SCR-002 (Critical) Unauthenticated arbitrary password reset (no token/ownership) -> account takeover; SCR-003 (Critical) Tokenless password reset keyed on sequential user id -> mass account takeover.

All findings were produced by static analysis only (no code was executed against the target) and verified for reachability before inclusion. Severity is scored with CVSS 3.1 and mapped to CWE and the OWASP Top 10.

Methodology

The review was conducted by a multi-agent pipeline. A reconnaissance stage detected the technology stack and mapped the attack surface; specialist stages then analyzed injection, access-control, and dependency/supply-chain risks in parallel against that map. A verification stage triaged every candidate for reachability and exploitability, culling false positives with code evidence and assigning CVSS 3.1 scores. Analysis was strictly static: the target's own code was read and modeled, never executed. This report is rendered deterministically from a single machine-readable findings record.

Scope & Coverage

Reviewed	app/Http/Controllers (85+ controllers: NGO, Front, Admin, Api, Auth, Payment), app/Http/Middleware (JwtMiddleware, AdminRoleMiddleware), app/Traits (MpdfTrait, PaymentTrait), routes/ (web.php ~710 routes, api.php, ad/ao/dg/director/uda desk files, fd1-frontend/fd7/fd9/fd91/renew), app/Imports (Fd6/Fd7 Excel importers), app/Models, app/Jobs, app/Helpers/helpers.php, config/ (auth, jwt, sonali-payment), resources/views (Blade PDF templates), composer.lock (SCA), public/ tree (uploads, bundled JS)
Not reviewed	vendor/ and node_modules/ (third-party; covered via SCA), app/Http/Controllers/sonali-payold.php (legacy dead code, route commented out), runtime data / storage / DB contents
Notes	Static analysis only; no code executed. Three parallel auth systems (session web + admin guards; JWT on /api). Public self-registration (POST /postRegistration, no middleware) makes every 'authenticated user' finding anonymous-reachable via one register step. nginx docroot = app root with unrestricted \.php\$ handler and no deny rule on public/uploads/, so any .php landing there executes. No committed .env in this repo (unlike prior reviews); secrets exposure is via .env.save*/swap files/hardcoded creds.

Findings Index

ID	Sev	CVSS	Title
SCR-001	Critical	9.8	Unauthenticated PDF/PHP-polyglot upload -> remote code execution
SCR-002	Critical	9.8	Unauthenticated arbitrary password reset (no token/ownership) -> account takeover
SCR-003	Critical	9.8	Tokenless password reset keyed on sequential user id -> mass account takeover
SCR-004	High	8.8	Unrestricted file upload -> PHP RCE via query-note reply attachments (4 sinks, one root cause)
SCR-005	High	8.1	API profileEdit/profileUpdate IDOR -> arbitrary-user disclosure and account takeover
SCR-006	High	7.7	Blind SSRF via mPDF remote resource fetch in PDF rendering
SCR-007	High	7.5	Unauthenticated IDOR: mass download of other NGOs' documents and national-ID scans
SCR-008	High	7.5	World-readable full database dump served from the web root
SCR-009	High	7.2	Any-admin -> superadmin privilege escalation (missing role gate on write endpoints)
SCR-010	Medium	6.5	Payment bypass: response() ignores verification and unconditionally sets paid flag

ID	Sev	CVSS	Title
SCR-011	Medium	6.5	DG-level application data readable by any admin role (missing admin.role gate)
SCR-012	Medium	6.1	Laravel framework 9.52.21 vulnerable to CRLF injection in default email validation (CVE-2026-48019)
SCR-013	Medium	5.3	Payment integrity: no amount validation and no-op updateLicenseFee stub on unauthenticated IPN
SCR-014	Medium	4.3	Stored HTML/content injection into official government PDFs
SCR-015	Low	4.0	Secrets and hardcoded credentials exposed by location (merged: IPN cred + committed secrets)
SCR-016	Low	4.2	setasign/fpdf 2.6.4 vulnerable (GHSA-2mgw-7q6p-8grg, fixed 2.6.7)
SCR-017	Low	3.7	symfony/mime 6.4.30 vulnerable (GHSA-qpmx-3rfj-7rhv, fixed 6.4.40)
SCR-018	Low	3.1	dompdf/dompdf 3.1.4 vulnerable (6 CVEs, fixed 3.1.6)
SCR-019	Low	4.2	Outdated bundled jQuery 3.2.1 with known XSS advisories (CVE-2020-11022/11023)

Detailed Findings

Unauthenticated PDF/PHP-polyglot upload -> remote code execution SCR-001

Critical

Severity	Critical
CVSS	3.1 9.8
Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-434 Unrestricted Upload of File with Dangerous Type, CWE-646 Reliance on File Name or Extension
OWASP	A01:2021 Broken Access Control, A03:2021 Injection
Category	Unrestricted File Upload / RCE
Location	app/Http/Controllers/NGO/Fd6FormController.php:3525 app/Http/Controllers/NGO/Fd6FormController.php:3530 app/Http/Controllers/NGO/Fd6FormController.php:3550 routes/web.php:1211
Found by	injection+verifier
Description	ministryLetterAnswerStore validates 'attachment.*' => 'nullable mimes:pdf max:5120'. Laravel's mimes rule checks file CONTENT via finfo, not the client extension; the sink saves time().'_'. \$file->getClientOriginalName(), preserving the attacker's filename. A file with valid %PDF magic bytes named shell.php passes validation and is written as <ts>_shell.php.
Impact	Arbitrary PHP execution as the web user, unauthenticated. The route sits after the auth group closes (group opens web.php:472, closes web.php:1206; this route is at web.php:1211). Target id is base64_decode(\$id) of a sequential ExternalMinistryNote id (trivially enumerable), no ownership check.
Reachability	Fully unauthenticated. Requires a guessable ministry-note id (base64 of small int). Confirmed reachable by verifier.
Evidence	Validation at Fd6FormController.php:3530 uses mimes:pdf (content-based); sink at :3550 saves getClientOriginalName(); route web.php:1211 is outside the auth Route::group. public/uploads/ has no PHP-exec deny under the stated nginx config.
Remediation	Generate the stored filename server-side and derive the extension from content (never getClientOriginalName()); store outside the web root and serve via an authenticated controller; add an extensions:pdf rule alongside mimes; deny .php execution under /uploads at nginx. <i>Pattern:</i> Never trust the client-supplied filename/extension. Validate content AND pin the stored extension; keep user uploads off the docroot.

Remediation (code)

```
$request->validate(['attachment'=>['required','file','mimes:pdf','extensions:pdf','max:5120']]);
$ext=$file->extension();
$name=time().'_'.Str::random(20).'.'.$ext;
$path=$file->storeAs('ministry_notes',$name,'local'); // storage/app, non-public
// nginx: location ~ /uploads/ { location ~ \.php$ { return 403; } }
```

Unauthenticated arbitrary password reset (no token/ownership) -> account takeover

SCR-002

Critical

Severity	Critical
CVSS	3.1 9.8
Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-640 Weak Password Recovery Mechanism, CWE-306 Missing Authentication for Critical Function
OWASP	A07:2021 Identification and Authentication Failures, A01:2021 Broken Access Control
Category	Broken Authentication / Weak Password Recovery
Location	app/Http/Controllers/Auth/AuthController.php:376 routes/web.php:382
Found by	authz+verifier
Description	POST /force-password-reset is a bare route with no middleware. Handler resolves the target as User::where('email',\$request->email)->orWhere('registration_number_given_by_admin', auth()->user()->registration_number_given_by_admin)->firstOrFail() then \$user->password=Hash::make(\$request->password); \$user->save(). No token, OTP, current-password, or session-ownership check.
Impact	Any attacker sets any NGO/applicant account's password to a known value and logs in (full takeover of that user and their submission data). For anonymous callers the ?-> yields null so the orWhere degenerates to registration_number IS NULL; the email clause still resolves a chosen victim.
Reachability	Unauthenticated. Hits the User (web) guard, not the separate admin guard. Confirmed by verifier.
Evidence	Route web.php:382 has no middleware (immediately after the AuthController group closes). AuthController.php:376-415 sets password with no verification token.
Remediation	Delete this endpoint or replace it with Laravel's signed, single-use, time-limited token flow (Password::sendResetLink / Password::reset). Never resolve the reset target from an unauthenticated request body. <i>Pattern:</i> Password recovery must prove control of the account (emailed single-use token) before changing credentials.

Remediation (code)

```

Password::reset($request->only('email','password','password_confirmation','token'),
    fn($user,$pw)=>$user->forceFill(['password'=>Hash::make($pw)])->save());

```


Tokenless password reset keyed on sequential user id -> mass account takeover

SCR-003

Critical

Severity	Critical
CVSS	3.1 9.8
Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-640 Weak Password Recovery Mechanism, CWE-639 Authorization Bypass Through User-Controlled Key, CWE-306 Missing Authentication for Critical Function
OWASP	A01:2021 Broken Access Control, A07:2021 Identification and Authentication Failures
Category	Broken Authentication / IDOR
Location	app/Http/Controllers/Auth/AuthController.php:51 app/Http/Controllers/Auth/AuthController.php:73 routes/web.php:366
Found by	authz+verifier
Description	passwordChangeConfirmed does User::find(\$request->id) then sets password from the request. The reset link generator passwordResetPage(\$id) (:73) carries a bare sequential user id with no signed token. Route web.php:366 has no middleware.
Impact	Attacker POSTs id=1,2,3,... plus a chosen password to reset every account in a loop. No token, ownership, or rate limit observed.
Reachability	Unauthenticated; distinct route/method from SCR-002. Enables bulk enumeration takeover. Confirmed by verifier.
Evidence	AuthController.php:51-68 find()+save(); reset link uses bare id at :73. Route in the no-middleware controller group (web.php:357-373).
Remediation	Replace the id-based link with a signed URL or hashed reset token stored in password_reset_tokens; validate and expire it server-side; rate-limit. <i>Pattern:</i> Never authorize a state change from a guessable, unsigned identifier.

Remediation (code)

```
// Use signed, single-use tokens (Password::reset) as in SCR-002; never trust $request->id.
```

Unrestricted file upload -> PHP RCE via query-note reply attachments (4 sinks, one root cause)

SCR-004

High

Severity	High
CVSS	3.1 8.8
Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-434 Unrestricted Upload of File with Dangerous Type, CWE-668 Exposure of Resource to Wrong Sphere
OWASP	A01:2021 Broken Access Control, A05:2021 Security Misconfiguration
Category	Unrestricted File Upload / RCE
Location	app/Http/Controllers/NGO/Fd6FormController.php:3433 app/Http/Controllers/NGO/Fd7FormController.php:1194 app/Http/Controllers/NGO/RenewController.php:1339 app/Http/Controllers/Front/Fd1NewController.php:1405 routes/web.php:749
Found by	injection+verifier
Description	Four byte-identical sinks: \$filename = time().'_'. \$file->getClientOriginalName(); \$file->move(public_path('uploads/external_ministry_notes'), \$filename); with no mimes/extensions/size validation. queryNoteReplyStore (Fd6 web.php:749, Fd7 web.php:755), renewQueryNoteReplyStore (web.php:734), registrationQueryNoteReplyStore (fd1-frontend.php:58).
Impact	A shell.php uploaded here lands in a web-served dir and executes under nginx -> full RCE as the web user. The query_note_id is not ownership-checked, so any registered user can target any note.
Reachability	Requires an authenticated User account, satisfied by public self-registration (POST /postRegistration) -> effectively anonymous-reachable (borderline 9.8; scored PR:L for the register step).
Evidence	Identical getClientOriginalName()->move(public_path(...)) pattern at all four locations; no validation in the methods. public/uploads/ has no PHP-exec deny.
Remediation	Same secure pattern as SCR-001: generate the filename, derive extension from content, storeAs outside the docroot, add a strict mimes/extensions allow-list, and deny .php under /uploads at nginx. <i>Pattern:</i> One shared upload helper that regenerates names and pins safe extensions; never write user files under public/.

Remediation (code)

```
$request->validate(['attachment'=>['required','file','mimes:pdf,jpg,jpeg,png','max:5120']]);
$name=time().'_'.Str::random(20).'.'. $file->extension();
$model->attachment=$file->storeAs('external_ministry_notes',$name,'local');
```

API profileEdit/profileUpdate IDOR -> arbitrary-user disclosure and account takeover

SCR-005

High

Severity	High
CVSS	3.1 8.1
Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
CWE	CWE-639 Authorization Bypass Through User-Controlled Key, CWE-284 Improper Access Control
OWASP	A01:2021 Broken Access Control
Category	Broken Object-Level Authorization
Location	app/Http/Controllers/Api/AuthController.php:253 app/Http/Controllers/Api/AuthController.php:280 routes/api.php:35
Found by	authz+verifier
Description	jwt.verify authenticates the token but the controllers discard the principal and operate on User::find(\$request->id). profileEdit?id=N returns the entire victim User row (incl. password hash); profileUpdate with id=N and the victim email (learned from profileEdit) enters the update branch and overwrites user_name/non_verified_email/password.
Impact	Any holder of a valid JWT (obtainable by anyone via public ngoRegister+ngoLogin) can read any account's data and take it over. The email-match guard is defeated by the step-1 disclosure.
Reachability	Any authenticated JWT user (public registration). Confirmed reliable ATO primitive by verifier.
Evidence	Api/AuthController.php:253 (profileEdit) and :280 (profileUpdate) trust \$request->id; middleware only checks token presence. Routes api.php:35-36.
Remediation	Operate only on the token's own user: replace \$request->id with JWTAuth::parseToken()->authenticate()->id in both methods. <i>Pattern:</i> Derive the acting user's identity from the authenticated session/token, never from a client-supplied id.

Remediation (code)

```
$user = auth('api')->user(); // or JWTAuth::parseToken()->authenticate();
$user->update($request->only('user_name','email'));
```

Blind SSRF via mPDF remote resource fetch in PDF rendering

SCR-006

High

Severity	High
CVSS	3.1 7.7
Vector	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N
CWE	CWE-918 Server-Side Request Forgery
OWASP	A10:2021 Server-Side Request Forgery
Category	Server-Side Request Forgery
Location	app/Traits/MpdfTrait.php:1 app/Http/Controllers/NGO/Fd3FormController.php:689 resources/views/front/fd3Form/fd3PdfView.blade.php:228
Found by	injection+verifier
Description	MpdfTrait sets no remote-fetch restriction; mPDF 8.2.7 fetches remote /CSS url() by default. Stored FD3 fields (purpose_of_donation, objectives_of_the_organization, project_account_details, purpose_details, money_received_details, method_details, equipment_details) are emitted unescaped via {!! !!} into WriteHTML.
Impact	A self-registered user stores (or an internal host); on PDF generation the server issues the request. Blind primitive (metadata is text) but internal port-scan via timing/error and hitting unauthenticated internal endpoints/cloud metadata are feasible. Scope:Changed.
Reachability	The submitting NGO user can trigger their own PDF render (fd3pdfview route under web auth); admin reviewers also render the same data.
Evidence	No enableRemoteFetch/restricting HTTP client in MpdfTrait; ~22 WriteHTML sinks, >=7 blades emit user DB fields via {!! !!}. fd3pdfview render at Fd3FormController.php:682->689.
Remediation	Escape user data in PDF blades ({{ }} not {!! !!}) or HTMLPurify with no external resource URLs; inject an mPDF HTTP client that blocks private/link-local ranges and non-http schemes; block egress from the render worker. <i>Pattern:</i> Treat any HTML that reaches a server-side renderer as untrusted; deny outbound requests to internal/link-local addresses.

Remediation (code)

```
$mpdf->setHttpClient(new BlockPrivateNetworkClient()); // rejects 169.254/10/127/172.16/192.168 +
non-http
// blade: {{ $fd3FormList->purpose_details }} (escaped)
```

Unauthenticated IDOR: mass download of other NGOs' documents and national-ID scans

SCR-007

High

Severity	High
CVSS	3.1 7.5
Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
CWE	CWE-639 Authorization Bypass Through User-Controlled Key
OWASP	A01:2021 Broken Access Control
Category	Broken Object-Level Authorization
Location	app/Http/Controllers/NGO/NamechangeController.php:850 app/Http/Controllers/NGO/NamechangeController.php:892 app/Http/Controllers/NGO/NamechangeController.php:907 app/Http/Controllers/NGO/NamechangeController.php:921 app/Http/Controllers/NGO/NamechangeController.php:940 routes/web.php:298
Found by	authz+verifier
Description	The NamechangeController route group (web.php:298-306) has no middleware. Each method does DB::table(...)->where('id',\$id)->value(<path_col>) then file_get_contents(public_path('/').\$path) and streams the bytes, with no ownership/session check; {id} is a plain integer.
Impact	Anonymous attacker loops GET /ngoMemberDocument/1..N (national-ID copies), /sourceOfFund/{id} (funding letters), /ngoOtherDocument/{id}, /otherPdfFromFDOneForm/{id}, /nameChangeDocDownload/{id} to exfiltrate every NGO's documents. High-value PII.
Reachability	Fully unauthenticated. Path-traversal secondary correctly dismissed (server-generated DB paths, no ../); this stands as pure broken object-level authz. Confirmed by verifier.
Evidence	Route group web.php:298-306 zero middleware; value(path)->file_get_contents streaming with no authz at NamechangeController.php:850/892/907/921/940.
Remediation	Wrap the group in auth + is_verify_email and add an ownership check joining the document's parent record to Auth::id() before serving. <i>Pattern:</i> Every object fetched by id must be authorized against the current principal (ownership or role), not merely authenticated.

Remediation (code)

```
Route::middleware(['auth'])->group(function(){ /* download routes */ });
// in each method: abort_unless($doc->ngo->user_id === auth()->id(), 403);
```

World-readable full database dump served from the web root

SCR-008

High

Severity	High
CVSS	3.1 7.5
Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
CWE	CWE-538 Insertion of Sensitive Information into Externally-Accessible File, CWE-522 Insufficiently Protected Credentials
OWASP	A01:2021 Broken Access Control, A05:2021 Security Misconfiguration
Category	Sensitive Data Exposure
Location	public/uploads/FdOneForm/2024-27-0317115230534910498380.sql:1 public/uploads/FdOneForm/2024-10-0317100601992640092317.sql:1
Found by	deps+verifier
Description	A ~4 MB phpMyAdmin dump sits under the public docroot with no auth gate, web-servable at /uploads/FdOneForm/2024-27-...498380.sql. Confirmed contents (redacted): 162 CREATE TABLE, INSERT for users/admins/oauth_clients/oauth_personal_access_clients, 72 bcrypt (\$2y\$) password hashes, and Passport OAuth client secrets. A second smaller dump is also present.
Impact	Direct GET yields full schema + all account hashes + OAuth client secrets. Offline cracking of the bcrypt hashes and reuse of Passport secrets escalates to account/system compromise.
Reachability	Unauthenticated HTTP GET. Confirmed by verifier.
Evidence	Both .sql files exist under public/uploads/FdOneForm/; dump contains users/admins/oauth_clients INSERTs and 72 bcrypt hashes (verified by location, not echoed).
Remediation	Delete both .sql files from public/ and purge from backups/VCS; rotate all exposed secrets (Passport client secrets, any embedded DB creds) and force password resets given hash exposure; block dump extensions at nginx; never write DB exports under public/. <i>Pattern:</i> Database exports and backups must never live under a web-served directory.

Remediation (code)

```
# nginx
location ~* \.(sql|sqlite|bak|gz|zip|env)$ { return 403; }
```

Any-admin -> superadmin privilege escalation (missing role gate on write endpoints)

SCR-009

High

Severity	High
CVSS	3.1 7.2
Vector	AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-862 Missing Authorization, CWE-269 Improper Privilege Management
OWASP	A01:2021 Broken Access Control
Category	Missing Authorization / Privilege Escalation
Location	routes/web.php:104 routes/web.php:117 app/Http/Controllers/Admin/AdminController.php:276
Found by	authz+verifier
Description	user.update/user.store/user.toggleStatus (web.php:104-107) sit inside the auth:admin group (opens web.php:93) but ABOVE the admin.role:superadmin group (opens web.php:117). The update() sink upserts admin_roles.role_id with only exists:roles,id validation, with no check that the actor is superadmin or that the target role isn't superadmin. Read-side endpoints ARE superadmin-gated -> protection applied inconsistently.
Impact	Any authenticated admin (any role) PUTs /admin/admin-user/{ownId} with role_id=<superadmin> and becomes superadmin; user.store lets any admin mint new admin accounts.
Reachability	Requires an existing admin credential (no admin self-registration; postRegistration creates web-guard users, not admins). Insider / compromised-low-priv-admin escalation -> PR:H.
Evidence	Routes at web.php:104-107 outside the admin.role:superadmin group (web.php:117); sink AdminController.php:276-284 updateOrCreate(['admin_id'=>\$id],['role_id'=>\$request->role_id]).
Remediation	Move the three write routes inside the admin.role:superadmin group; defense-in-depth: reject a privileged role_id unless the actor is superadmin. <i>Pattern:</i> Apply the same authorization gate to writes as to the reads they mirror; never let a lower tier assign a higher tier.

Remediation (code)

```
Route::middleware('admin.role:superadmin')->group(function(){ /* user.update/store/toggleStatus */
});
if(Role::find($request->role_id)?->name==='superadmin' && !auth('admin')->user()->hasRole('superadmin')) abort(403);
```

Payment bypass: response() ignores verification and unconditionally sets paid flag

SCR-010

Medium

Severity	Medium
CVSS	3.1 6.5
Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:L
CWE	CWE-345 Insufficient Verification of Data Authenticity, CWE-639 Authorization Bypass Through User-Controlled Key, CWE-476 NULL Pointer Dereference
OWASP	A04:2021 Insecure Design
Category	Insufficient Verification / Business Logic
Location	app/Http/Controllers/PaymentController.php:391 app/Http/Controllers/PaymentController.php:407 routes/web.php:446
Found by	authz+verifier
Description	response() (route web.php:446, no auth) calls verifyPayment() at :407 and discards its return, then unconditionally runs FormCompleteStatus::where('id',...)->update(['form_eight_status'=>1]) for Auth::user()->id when type=='registration'. The lookup at :403 uses ->where('token',\$token) but the column is 'Token', so \$transaction is usually null and verification is skipped entirely.
Impact	An authenticated user calls GET /payment/response?Token=anything -> registration Form-8 marked paid without any payment. Flag is keyed on session user, not the payment owner; null-deref on the unauth path is a secondary 500/DoS.
Reachability	Authenticated NGO user (public registration); no payment required. Confirmed by verifier.
Evidence	PaymentController.php:407 return value discarded; :412-416 unconditional update; :403 wrong column case ('token' vs 'Token').
Remediation	Require verified Status==200 AND an amount match before setting the flag; bind the flag to the transaction's user_id (not the session), use the correct column, and add auth/signed token to the route. <i>Pattern:</i> Grant paid state only after server-side verification of authenticity AND amount, bound to the payment's owner.

Remediation (code)

```
$txn=DB::table('sonali_payments')->where('Token',$token)->first(); abort_unless($txn,404);
$v=$this->getVerifyPaymentResponse($token); abort_unless(($v->Status??null)=='200',402);
abort_unless($this->amountMatches($txn,$v),402);
FormCompleteStatus::where('user_id',$txn->user_id)->update(['form_eight_status'=>1]);
```


DG-level application data readable by any admin role (missing admin.role gate)

SCR-011

Medium

Severity	Medium
CVSS	3.1 6.5
Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
CWE	CWE-862 Missing Authorization
OWASP	A01:2021 Broken Access Control
Category	Missing Authorization
Location	routes/dg.php:127 app/Http/Controllers/Admin/DGCommonRouteController.php:1
Found by	authz+verifier
Description	A second route group in dg.php (lines 127-137) requires only auth:admin with no admin.role child, while the sibling group at dg.php:14 correctly nests admin.role:dg. DGCommonRouteController methods (formNoFour/Five/Seven, fc2/fd3/fd4/fd5 forms) return all applications or fetch an arbitrary base64_decode(\$id).
Impact	Any authenticated admin (any staff role) reads DG-tier registration/renewal form data and enumerates arbitrary application ids. Insider horizontal/vertical read bypass; base64(id) is obfuscation, not authorization.
Reachability	Requires an existing admin account (no admin self-registration). Insider threat. Confirmed by verifier.
Evidence	dg.php:127-137 group lacks admin.role, confirmed by comparison with the guarded sibling group at dg.php:14; *Show(\$id) does base64_decode then fetch.
Remediation	Wrap the second dg.php group in the appropriate admin.role middleware matching the least privilege that should see DG form data. <i>Pattern:</i> Every admin route group carries an explicit role gate; never rely on a sibling group's gate.

Remediation (code)

```
Route::middleware(['auth:admin','admin.role:dg'])->group(function(){ /* DGCommonRouteController
    routes */ });
```

Laravel framework 9.52.21 vulnerable to CRLF injection in default email validation (CVE-2026-48019)

SCR-012

Medium

Severity	Medium
CVSS	3.1 6.1
Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
CWE	CWE-93 Improper Neutralization of CRLF Sequences
OWASP	A06:2021 Vulnerable and Outdated Components
Category	Vulnerable Dependency
Location	composer.lock:1
Found by	deps+verifier
Description	Installed laravel/framework v9.52.21 (< patched 10.48.29 backport). The default email validation rule accepts CRLF-bearing addresses that can reach the mailer as header injection.
Impact	User email from public registration/contact input flows into Mail::send(...->to(\$request->email)) -> header-injection surface.
Reachability	Reachable via public registration/contact email fields.
Evidence	composer.lock pins laravel/framework 9.52.21; advisory CVE-2026-48019.
Remediation	Upgrade laravel/framework to the patched release (9.x backport / 10.48.29) or apply the email:strict / DNS-validated rule. <i>Pattern:</i> Keep the framework on a supported, patched line; validate email strictly.

Remediation (code)

```
'email' => 'required|email:strict,dns'
```

Payment integrity: no amount validation and no-op updateLicenseFee stub on unauthenticated IPN

SCR-013

Medium

Severity	Medium
CVSS	3.1 5.3
Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N
CWE	CWE-345 Insufficient Verification of Data Authenticity, CWE-840 Business Logic Errors
OWASP	A04:2021 Insecure Design
Category	Business Logic / Insufficient Verification
Location	app/Http/Controllers/PaymentController.php:311 app/Http/Controllers/PaymentController.php:450 routes/api.php:61
Found by	authz+deps+verifier
Description	sonaliIPNCheck (unauth, api.php:61) re-verifies server-to-server and only marks Success on bank Status=='200' (so forging 'paid' is mitigated - see dismissed SCR-D03), BUT performs no amount comparison against the expected fee, and updateLicenseFee() at :450 is a no-op stub (return true). The hardcoded IPN credential at :329 is reported separately as part of SCR-015 (secrets by location).
Impact	Any bank-200 transaction is accepted as full payment regardless of amount paid (underpayment risk if the gateway 200s partial settlements); the benefit-granting routine does nothing.
Reachability	Unauthenticated endpoint; forge-paid mitigated, residual is business-logic. Confirmed by verifier.
Evidence	PaymentController.php:347 re-verify; no amount check anywhere in the handler; :450-453 updateLicenseFee(){ return true; }.
Remediation	Validate the verified paid amount against the expected fee before accepting payment; implement updateLicenseFee idempotently bound to the transaction's verified amount. <i>Pattern:</i> Verify BOTH authenticity and the exact expected amount for every payment callback.

Remediation (code)

```
$expected=$this->expectedFeeFor($payment->serviceId);
if((float)$verifyResponse->PaidAmount !== (float)$expected) throw new \Exception('Amount mismatch');
```

Stored HTML/content injection into official government PDFs

SCR-014

Medium

Severity	Medium
CVSS	3.1 4.3
Vector	AV:N/AC:L/PR:L/UI:R/S:U/C:N/I:L/A:N
CWE	CWE-79 Improper Neutralization of Input During Web Page Generation, CWE-116 Improper Encoding or Escaping of Output
OWASP	A03:2021 Injection
Category	Improper Output Encoding
Location	resources/views/front/fd3Form/fd3PdfView.blade.php:228 app/Http/Controllers/NGO/FdFiveFormController.php:554 app/Http/Controllers/NGO/Fd9Controller.php:799 app/Http/Controllers/Admin/AoController.php:331
Found by	injection+verifier
Description	The same unescaped {!! !!} user fields that enable SCR-006 also let an NGO user inject arbitrary HTML/content/layout (tables, replaced text, forged headings/seals via CSS) into official FD3/FD5/FD9 and admin ministry-note PDFs. mPDF executes no <script>, so this is document-integrity, not code exec.
Impact	Reviewers/DG office print these as authoritative documents; injected content can alter what a reviewer then signs -> document forgery/integrity risk.
Reachability	Any registered user stores the payload; a reviewer/admin (or the user) triggers generation (UI:R).
Evidence	Unescaped {!! !!} emission of stored user fields into WriteHTML at the listed sinks.
Remediation	Escape with {{ }} or HTMLPurifier with a minimal allow-list; never emit stored user fields via {!! !!} in PDF templates. <i>Pattern:</i> Output-encode all user data at the rendering boundary; PDFs are output too.

Remediation (code)

```
{{ $fd3FormList->purpose_details }} {{-- escaped, no raw {!! !!} --}}
```

Secrets and hardcoded credentials exposed by location (merged: IPN cred + committed secrets)

SCR-015

Low

Severity	Low
CVSS	3.1 4.0
Vector	AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
CWE	CWE-798 Use of Hard-coded Credentials, CWE-522 Insufficiently Protected Credentials, CWE-540 Inclusion of Sensitive Information in Source Code, CWE-315 Cleartext Storage of Sensitive Information
OWASP	A05:2021 Security Misconfiguration, A07:2021 Identification and Authentication Failures
Category	Credential Exposure
Location	app/Http/Controllers/PaymentController.php:329 config/sonali-payment-config.php:10 .env.save:1 .env.save.1:1 .env.example:1 routes/.web.php.swp:1
Found by	deps+authz+verifier
Description	Consolidated secrets-by-location finding (per orchestrator merge of F3a's hardcoded IPN credential with the deps secrets item). Hardcoded Sonali IPN basic-auth credential at PaymentController.php:329 (Username 'ngoabsonaliipn' / 17-char Password, redacted); committed default test-mode-token at config/sonali-payment-config.php:10; env copies .env.save/.env.save.1/.env.example carrying reCAPTCHA secrets and setting APP_DEBUG=true/APP_ENV=local; committed editor swap files ..env.swp and routes/.web.php.swp[.save] (may leak source + secrets). No live application .env is committed in this repo.
Impact	Anyone with source/repo access holds the IPN shared secret (direct-impact reduced because the IPN also re-verifies with the bank - see SCR-013), reCAPTCHA secrets, and potentially leaked source from swap files. APP_ENV=local in the committed env copies also gates other misconfig (e.g. log-viewer default policy opening).
Reachability	Requires source/tree access (AV:L). Rotate regardless.
Evidence	Grep/gitLeaks/trufflehog confirmed the hardcoded cred at PaymentController.php:329 and the listed committed secret files (reported by location, redacted).
Remediation	Move all credentials to environment/secret storage (config() + env), rotate the IPN credential and reCAPTCHA secrets, delete .env.save*/.env.example-with-secrets and all editor swap files from the tree, and add them to .gitignore; use hash_equals for credential comparison. <i>Pattern:</i> No secret in source; compare secrets in constant time; keep only .env.example with placeholder values.

Remediation (code)

```
if(!hash_equals(config('sonali.ipn_pass'),$request->Password)) abort(401);  
// .gitignore: .env.save* *.swp .env
```

setasign/fpdi 2.6.4 vulnerable (GHSA-2mgw-7q6p-8grg, fixed 2.6.7)

SCR-016

Low

Severity	Low
CVSS	3.1 4.2
Vector	AV:L/AC:L/PR:L/UI:R/S:U/C:L/I:L/A:L
CWE	CWE-1035 Vulnerable Third-Party Component
OWASP	A06:2021 Vulnerable and Outdated Components
Category	Vulnerable Dependency
Location	<code>composer.lock:1</code>
Found by	deps+verifier
Description	Installed setasign/fpdi v2.6.4 (the candidate's '6.0' was incorrect; 2.6.4 < 2.6.7 is genuinely vulnerable). Reachable only if a user-uploaded PDF passes through FPDI's import/parse path (mPDF is the primary engine).
Impact	Parser-level vulnerability triggered via crafted PDF import; conditional reachability.
Reachability	Conditional on a user-controlled PDF reaching FPDI import.
Evidence	<code>composer.lock</code> pins setasign/fpdi 2.6.4; advisory GHSA-2mgw-7q6p-8grg.
Remediation	Upgrade setasign/fpdi to $\geq 2.6.7$. <i>Pattern:</i> Track and patch transitive PDF libraries.

Remediation (code)

```
composer require setasign/fpdi:^2.6.7
```

symfony/mime 6.4.30 vulnerable (GHSA-qpmx-3rfj-7rhv, fixed 6.4.40) SCR-017 Low

Severity	Low
CVSS	3.1 3.7
Vector	AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N
CWE	CWE-1035 Vulnerable Third-Party Component
OWASP	A06:2021 Vulnerable and Outdated Components
Category	Vulnerable Dependency
Location	composer.lock:1
Found by	deps+verifier
Description	Installed symfony/mime v6.4.30 (< 6.4.40). Reachable via the mailer path (registration/notification emails).
Impact	Advisory-dependent MIME handling issue via the mail path.
Reachability	Via mailer (registration/notification emails).
Evidence	composer.lock pins symfony/mime 6.4.30; advisory GHSA-qpmx-3rfj-7rhv.
Remediation	Upgrade symfony/mime to >=6.4.40. <i>Pattern:</i> Keep Symfony components on the patched 6.4 line.

Remediation (code)

```
composer update symfony/mime
```


dompdf/dompdf 3.1.4 vulnerable (6 CVEs, fixed 3.1.6)

SCR-018

Low

Severity	Low
CVSS	3.1 3.1
Vector	AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N
CWE	CWE-1035 Vulnerable Third-Party Component
OWASP	A06:2021 Vulnerable and Outdated Components
Category	Vulnerable Dependency
Location	<code>composer.lock:1</code>
Found by	deps+verifier
Description	Installed dompdf/dompdf v3.1.4 (< 3.1.6). dompdf is referenced on user-facing NGO paths (NGO/ProfileController.php, NGO/RenewController.php) as well as admin, so it is not dead code. Exploiting the specific CVEs requires attacker-controlled HTML/CSS/SVG reaching the renderer; Blade auto-escaping reduces (not eliminates) this. Not traced to a concrete injection sink in this review.
Impact	Conditional; watch item alongside the mPDF SSRF/HTML-injection findings.
Reachability	Conditional on user-controlled HTML reaching loadHtml().
Evidence	<code>composer.lock</code> pins dompdf/dompdf 3.1.4; used in <code>NGO/ProfileController.php</code> , <code>NGO/RenewController.php</code> .
Remediation	Upgrade dompdf/dompdf to <code>>=3.1.6</code> and confirm no raw/user HTML is passed to loadHtml(). <i>Pattern:</i> Patch the renderer and keep user HTML out of it.

Remediation (code)

```
composer require dompdf/dompdf:^3.1.6
```

Outdated bundled jQuery 3.2.1 with known XSS advisories (CVE-2020-11022/11023)

SCR-019

Low

Severity	Low
CVSS	3.1 4.2
Vector	AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N
CWE	CWE-1035 Vulnerable Third-Party Component, CWE-79 Cross-site Scripting
OWASP	A06:2021 Vulnerable and Outdated Components
Category	Vulnerable Dependency
Location	public/front/assets/js/jquery_3.2.1.js:1
Found by	deps+verifier
Description	Bundled jQuery 3.2.1 (CVE-2020-11022/11023, fixed 3.5.0) is served by ~10 blade views (admin DG view-application and front ministryView pages). Exploitation requires attacker-controlled HTML flowing into a jQuery DOM sink (.html() etc.); not traced to a concrete sink here.
Impact	Component-hygiene finding; latent DOM-XSS if untrusted HTML reaches a jQuery manipulation sink.
Reachability	Conditional on a jQuery HTML sink receiving untrusted input.
Evidence	public/front/assets/js/jquery_3.2.1.js served by ~10 blades (confirmed).
Remediation	Upgrade the bundled jQuery to >=3.5.1. <i>Pattern:</i> Keep front-end libraries patched; prefer .text() over .html() for untrusted data.

Remediation (code)

```
// replace public/front/assets/js/jquery_3.2.1.js with jQuery 3.5.1+
```

Dependency & Supply-Chain Risks

Package	Version	Advisory	Fixed	Sev
laravel/framework	9.52.21	CVE-2026-48019	10.48.29	Medium
phpoffice/ phpspreadsheet	1.30.2	GHSA-q4q6-r8wh-5cgh; GHSA-87m4-826x-3crx	1.30.5	Low
setasign/fpdi	2.6.4	GHSA-2mgw-7q6p-8grg	2.6.7	Low
symfony/mime	6.4.30	GHSA-qpmx-3rfj-7rhv	6.4.40	Low
dompdf/dompdf	3.1.4	CVE-2026-59943/ 59942/59941/56722/ 55555/55554	3.1.6	Low
guzzlehttp/guzzle	7.10.0	GHSA-v5mv-p594-2x33	7.15.2	Low
jquery (bundled)	3.2.1	CVE-2020-11022; CVE-2020-11023	3.5.1	Low

Investigated & Dismissed

- SCR-D01 — DISMISSED: phpspreadsheet 1.30.2 'critical' RCE advisories (GHSA-q4q6-r8wh-5cgh / GHSA-87m4-826x-3crx): Not exploitable as scored; upgrade to phpspreadsheet $\geq 1.30.5$ as hygiene.
- SCR-D02 — DISMISSED: F6 'fail-open' JWT middleware: No security action; optionally return HTTP 401 instead of 200 for correctness.
- SCR-D03 — DISMISSED: forge 'paid' state via unauthenticated Sonali IPN: No standalone action; residual issues tracked under SCR-013 and SCR-015.
- SCR-D04 — DISMISSED: SQL injection (orderByRaw / DB::raw / selectRaw / whereRaw): No action; keep using parameter bindings for any future raw SQL.
- SCR-D05 — DISMISSED: path traversal in file_get_contents(public_path().\$dbValue) downloaders: No traversal action; authz fix tracked under SCR-007. Optionally realpath-contains as defense-in-depth.
- SCR-D06 — DISMISSED: guzzlehttp/guzzle 7.10.0 host-check bypass (GHSA-v5mv-p594-2x33): Low priority; upgrade guzzle to $\geq 7.15.2$ during routine maintenance.
- SCR-D07 — DISMISSED: unlink() arbitrary-delete (13 sinks): No action; keep delete paths derived from server-generated storage keys.
- SCR-D08 — DISMISSED: ReDoS via non-literal regexp (7 SAST hits): No action.
- SCR-D09 — DISMISSED: JavaScript XSS execution inside mPDF-rendered PDFs: No JS-XSS action; escaping fix tracked under SCR-006 and SCR-014.
- SCR-D10 — DISMISSED: FdoneformController::ministryLetterAnswerStore upload: No action; this is the correct pattern the other upload sites should adopt.
- SCR-D11 — DISMISSED: singleStaffDetailsInformationEexecutiveUpdate upload sink: No action; remove the dead code to reduce confusion.
- SCR-D12 — DISMISSED (as standalone): 6-digit email-verification token: Hardening: add rate-limiting and short expiry to the email-verification code, or widen it.

References

CWE (<https://cwe.mitre.org/>), OWASP Top 10 (<https://owasp.org/Top10/>), CVSS 3.1 (<https://www.first.org/cvss/>).